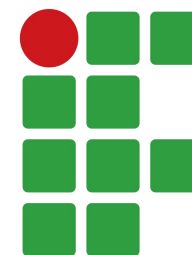


PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco

Objetivos

- Conhecer os principais conceitos relativos ao desenvolvimento de software voltado para dispositivos móveis.



Objetivos

- Conhecer os principais conceitos relativos ao desenvolvimento de software voltado para dispositivos móveis.

Android 

 Kotlin

Professor

Lucas Sampaio Leite

lucas.leite@belojardim.ifpe.edu.br



A disciplina

- Quando serão as aulas?
 - Segundas-feiras das 08h15 às 10h45
 - Terças-feiras das 07h30 às 10h00
- Carga Horária: 120ha/90hr
- Pré-requisito: P00



Ementa

- Compreensão das principais características sobre dispositivos móveis.
- Conhecimento das plataformas de desenvolvimento mais utilizadas.
- Entendimento da arquitetura de Software Móvel.
- Reconhecimento dos principais métodos de comunicação para Software móvel.

Conteúdo Programático

1. A linguagem Kotlin
2. Introdução ao Android
3. Configuração e uso do ambiente de desenvolvimento
4. Activity e o seu ciclo de vida
5. Ações, navegação e Intents
 - Tratamento de eventos de click
 - Transição entre activities
 - Passagens de parâmetros e objetos
 - Formas de navegação

Conteúdo Programático

5. Gerenciadores de Layout
6. ListView e RecyclerView
7. Persistência com banco de dados
 - Local com SQLite
 - Remota com Firebase
8. Conexão com WebServices
 - Retrofit
9. Fragmentos
10. Outros Componentes

Ferramentas e ambientes de desenvolvimento

- Kotlin Playground
- Android training courses
- Android Studio IDE
- Beecrowd



Metodologia

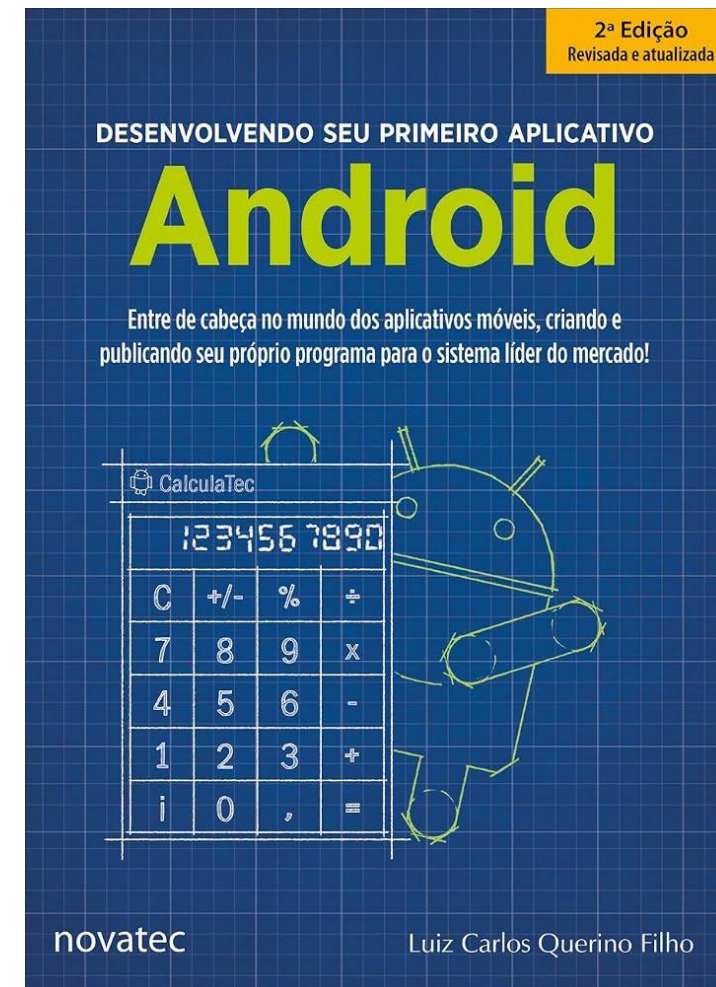
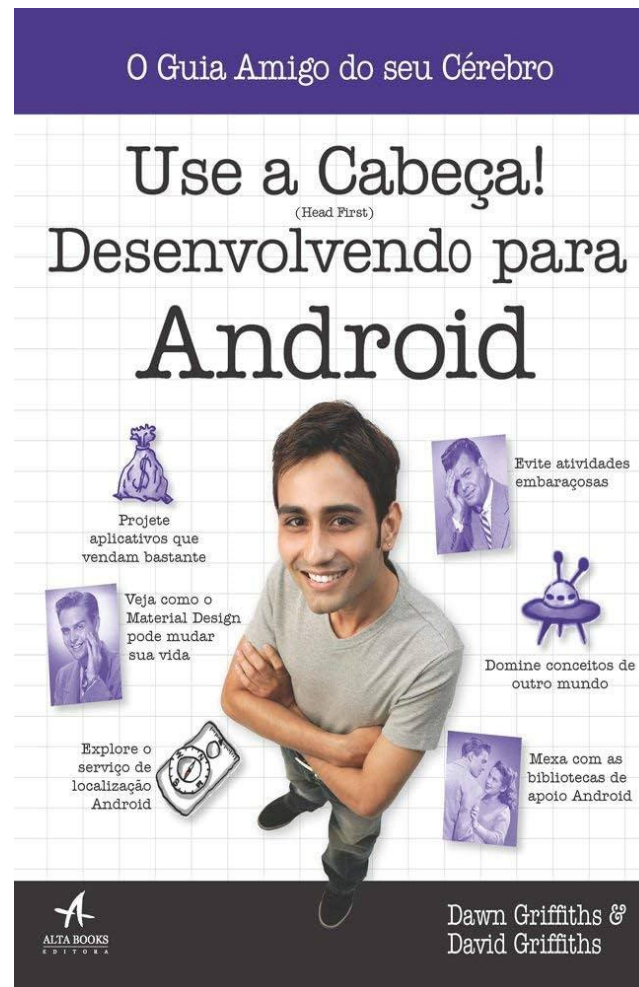
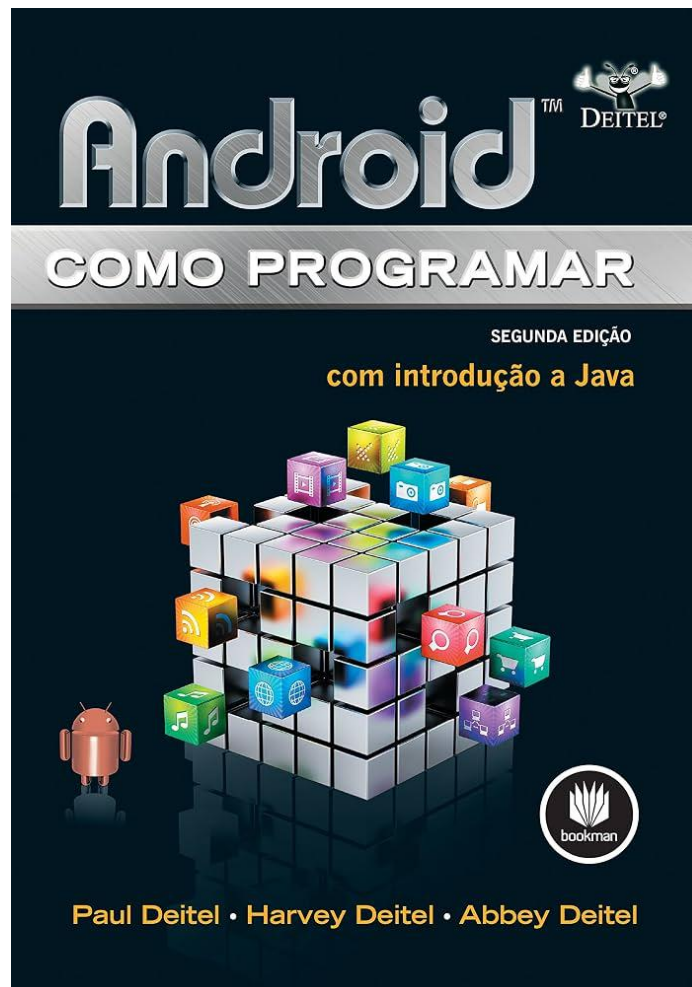
- Aulas síncronas e presenciais
 - Aulas expositivas e dialogadas
 - Abordagem prática
 - Aulas de resolução de exercícios
- Com chamada

Avaliação

- AV1
 - Lista de nivelamento: 30% da nota
 - Seminários: 70% da nota (avaliação individual e em grupo)
- AV2
 - Projeto da disciplina: 100% da nota (apresentação + acompanhamento)



Bibliografia Básica



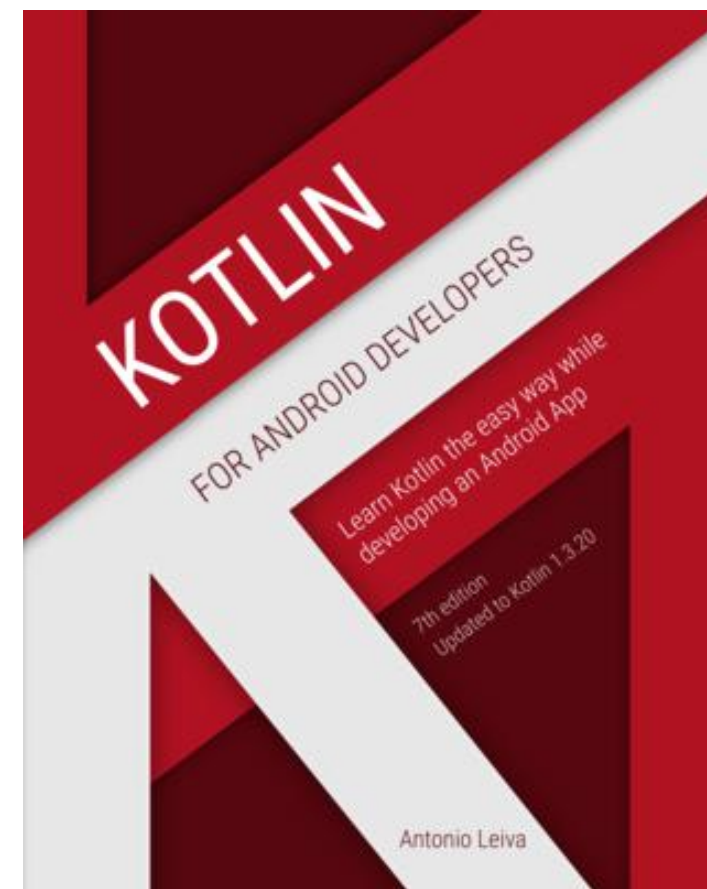
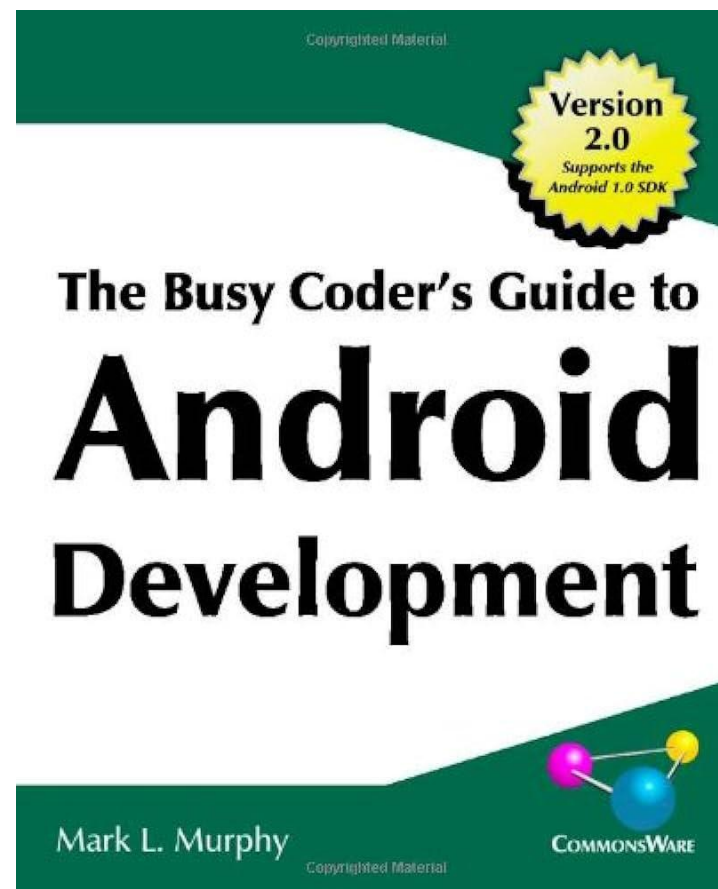
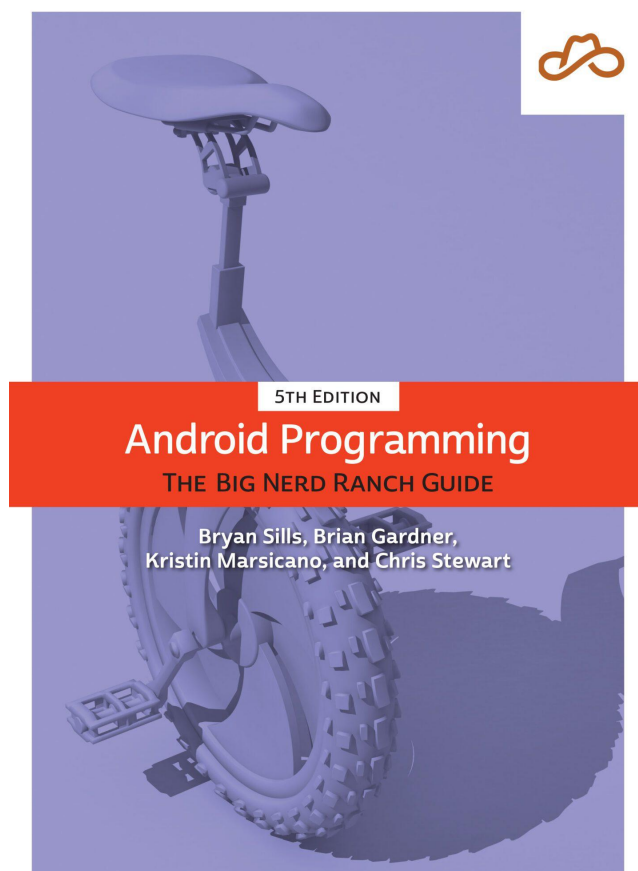
Bibliografia Básica

- DEITEL, Harvey M.; DEITEL, Paul J. ; DEITEL, Abbey. Android: Como Programar com Introdução a Java. 2. ed. Porto Alegre: Grupo A Educação, 2015.
- GRIFFITHS, David; GRIFFITHS, Dawn. Use a Cabeça! Desenvolvendo Para Android. 1. ed. Rio de Janeiro: Altabooks, 2016.
- QUERINO FILHO, Luiz Carlos. Desenvolvendo seu primeiro aplicativo Android. 2. ed. São Paulo: Novatec, 2017.

Bibliografia Complementar

- LECHETA, Ricardo R. Google Android: Aprenda a criar aplicações para dispositivos móveis com o o Android SDK. 2. ed. São Paulo: Universidade Católica Editora, 2010.
- JOHNSON, Thienne M. Java para Dispositivos Móveis: Desenvolvendo Aplicações em J2ME. São Paulo: Universidade Católica Editora, 2007.
- PANHALE, Mahesh. Beginning Hybrid Mobile Application Development. 1. ed. California: Apress, 2015.
- COELHO, Pedro. Desenvolvimento Móvel com HTML5: Integração com JavaScript, CSS3 e jQuery Mobile. 1. ed. Lisboa: FCA, 2015.
- HARMES, Dan. Desenvolvimento de Aplicativos Móveis com Xamarin. 1. ed. São Paulo: Novatec, 2015.

Outras referências



Outras referências (links oficiais dos livros indicados)

- <https://bignerdranch.com/books/android-programming-the-big-nerd-ranch-guide-5th-edition/>
- <https://commonsware.com/Android/>
- <https://leanpub.com/kotlin-for-android-developers>

Outras referências

- Documentação oficial do Kotlin:
<https://kotlinlang.org/docs/kotlin-reference.pdf>
- Documentação oficial do Android:
<https://developer.android.com/>
- Cursos oficiais do Android:
<https://developer.android.com/courses>

Referência de exercícios para Kotlin

[LOGIN](#)[REGISTER](#)[FORUM](#)[PROFESSORS](#)[CORPORATIONS](#)[beecrowd](#)

BEECROWD



We are a global community of developers committed to keep evolving as students and professionals. Train algorithms and programming challenges and become the expert you always dreamed to be.

[CORPORATE PAGE](#)

USE SOCIAL SIGN IN



OR

EMAIL

PASSWORD

☐ REMEMBER ME (7 DAYS)

[SIGN IN](#)

FIRST TIME HERE?

SIGN UP today to join one of the largest developer and competitive programming communities in the world!

COMPETITION AND RANKING

Join the brightest minds in competitive programming! Participate in competitions, contests and tournaments! Compare your knowledge with your peers. Level up, grow and shine in your career!



BEECROWD ACADEMIC

The beecrowd Academic is a module for Educational Institutions, Professors and Coaches. Here you can create courses, exercise lists and track your students progress giving them real-time feedback.

[ACCESS ACADEMIC](#)

<https://judge.beecrowd.com/>

Material de aula e informações sobre a disciplina



Prof. Lucas Sampaio Leite

Instituto Federal de Pernambuco - Campus Belo Jardim

Bem-vindo ao meu portfólio! Aqui você encontrará materiais atualizados e recursos das disciplinas que ministro no Instituto Federal de Pernambuco, preparados para apoiar sua aprendizagem de forma clara e acessível.

| Disciplinas

Curso Técnico Subsequente em Informática para internet

Eletricidade e eletrônica

Conteúdo e materiais atualizados.

Acessar

Engenharia de Software

Algoritmos e Estrutura de Dados

Conteúdo e materiais atualizados.

Acessar

Boas práticas

- Para a dinâmica de aprendizagem da disciplina funcionar, é importante realizar as práticas e exercícios passados em sala (não deixem acumular atividades).
- Os conteúdos dos seminários vão ajudar nos projetos.
- O projeto não deve ser deixado para última hora.
- Organize seu tempo entre as disciplinas!!!

Introdução ao Kotlin

- Kotlin é uma linguagem de programação moderna, mas já madura, projetada para deixar os desenvolvedores **mais felizes** .

Introdução ao Kotlin

- Kotlin é uma linguagem de programação moderna, mas já madura, projetada para deixar os desenvolvedores **mais felizes** .

```
public class Pessoa {  
    private String nome;  
    private int idade;  
  
    public Pessoa(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    public String getNome() { return nome; }  
    public int getIdade() { return idade; }  
}
```



```
class Pessoa(val nome: String, val idade: Int)
```

Introdução ao Kotlin

- Kotlin é uma linguagem de programação moderna, mas já madura, projetada para deixar os desenvolvedores **mais felizes** .
- Pode-se definir Kotlin como uma linguagem de programação pragmática que combina os paradigmas de Orientação a Objetos e Programação Funcional com foco em interoperabilidade, segurança, clareza e suporte a ferramentas.

Introdução ao Kotlin

- Criada pela JetBrains, empresa responsável pelo IntelliJ IDEA (base do Android Studio), como uma alternativa moderna ao Java.
- 2011: Kotlin é anunciada durante o JVM Language Summit.
- 2016: Lançamento da versão 1.0, a primeira versão estável, com suporte ao Android via plugin no Android Studio.
- 2017: Durante o Google I/O, Kotlin torna-se oficialmente uma linguagem suportada para o desenvolvimento Android.
- 2019: Kotlin torna-se a linguagem recomendada (primeira opção) para o desenvolvimento Android.

Introdução ao Kotlin

- Atualmente, mais de 50% dos desenvolvedores Android utilizam Kotlin como linguagem principal, enquanto cerca de 30% utilizam Java.
- Cerca de 70% dos desenvolvedores que usam Kotlin como linguagem principal afirmam que ela aumenta sua produtividade.

Introdução ao Kotlin

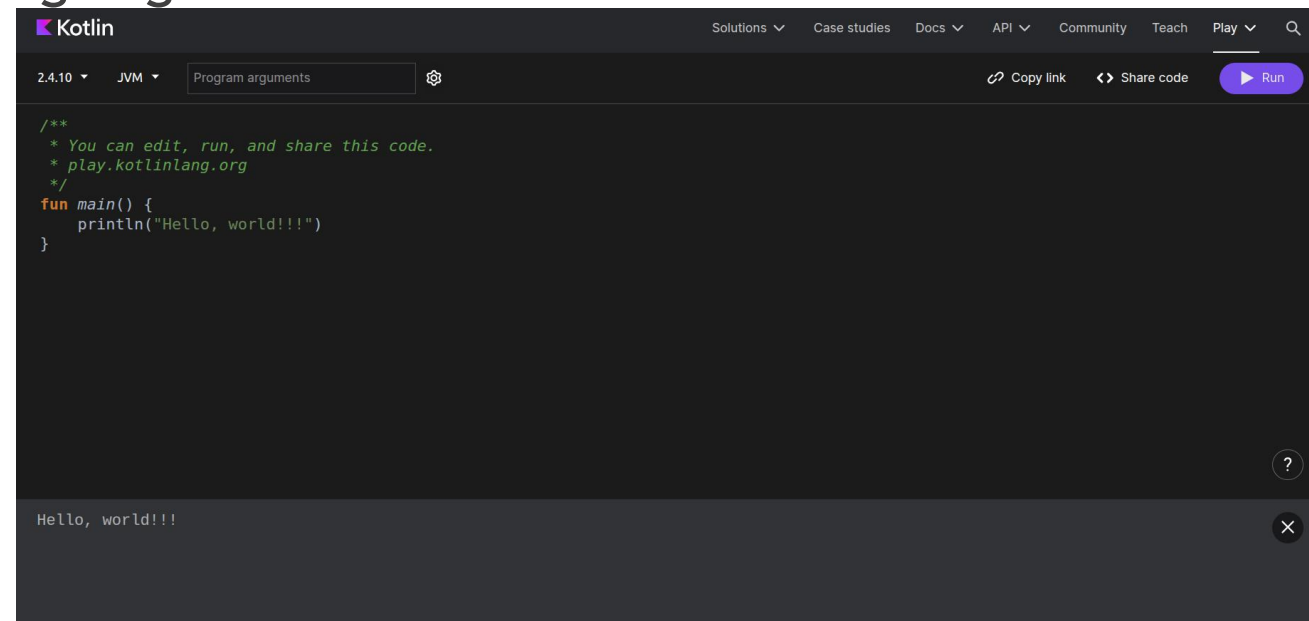
- Características do Kotlin:
 - Concisa: menos código para realizar as mesmas tarefas.
 - Segura: reduz erros comuns, como referências nulas (NullPointerException).
 - Interoperável: compatível com bibliotecas e códigos escritos em Java.
 - Multiplataforma: permite desenvolver aplicações Android, Web, Desktop e Servidor.

Introdução ao Kotlin

- Características do Kotlin:
 - Orientada a Objetos e Funcional: combina os dois paradigmas de programação.
 - Expressiva: código mais legível e de fácil manutenção.
 - Produtiva: recursos como inferência de tipos e classes de dados (data class) aceleram o desenvolvimento.
 - Excelente suporte a ferramentas: integração nativa com Android Studio e IntelliJ IDEA.

Introdução ao Kotlin

- Kotlin Playground:
 - Ambiente online e gratuito para escrever, executar e testar códigos Kotlin.
 - Não requer instalação ou configuração.
 - Ideal para aprender a linguagem, realizar experimentos e compartilhar exemplos de código.
 - Disponível em: <https://play.kotlinlang.org/>



```
Kotlin
Solutions Case studies Docs API Community Teach Play
2.4.10 JVM Program arguments Copy link Share code Run

/**
 * You can edit, run, and share this code.
 * play.kotlinlang.org
 */
fun main() {
    println("Hello, world!!!")
}

Hello, world!!!
```

Sintaxe básica do Kotlin

- Comentários em Kotlin:

```
// Comentário de uma linha

/*
    Comentário
    de múltiplas linhas
*/

/**
 * Comentário de documentação (KDoc)
 * Utilizado para documentar classes, funções e propriedades.
 */
fun saudar(nome: String) {
    println("Olá, $nome!")
}
```

Sintaxe básica do Kotlin

- Definindo variáveis:
 - Utiliza-se a palavra-chave `var` para declarar variáveis mutáveis.
 - O tipo pode ser informado explicitamente ou inferido pelo compilador.
 - Toda variável deve ser inicializada antes de ser utilizada.

```
var nome: String = "Lucas"  
var idade = 30           // Tipo inferido: Int  
var altura = 1.75        // Tipo inferido: Double
```

Sintaxe básica do Kotlin

- Definindo referências somente leitura:
 - Utiliza-se a palavra-chave `val` para declarar uma referência somente leitura.
 - Após a inicialização, seu valor não pode ser reatribuído.
 - Equivale ao modificador `final` em Java.

```
!   val nome = "Lucas"
!   val idade: Int = 30
!   nome = "Maria" // Erro de compilação
```



```
! 'val' cannot be reassigned.
```

Sintaxe básica do Kotlin

- Boas práticas: prefira val
 - É uma prática recomendada utilizar val sempre que a variável não precisar ter seu valor alterado.
 - Utilize var apenas quando for realmente necessário modificar o valor da variável.
 - O uso de val torna o código mais seguro, legível e fácil de manter.
 - Também reduz alterações acidentais durante o desenvolvimento.

Sintaxe básica do Kotlin

- Tipos básicos do Kotlin:

Tipo	Descrição
Double	Tipo numérico para valores de ponto flutuante, com <code>Double</code> é possível guardar valores com precisão bem grande pois ele ocupa 64 bits
Float	Tipo numérico para valores de ponto flutuante, com a diferença de que tem uma precisão menor que o <code>Double</code> pois ele ocupa 32 bits
Long	Usado para valores inteiros, a diferença dele para o <code>Int</code> é que, como ele ocupa 64 bits, você consegue representar números muito maiores
Int	Talvez um dos tipos numéricos mais comuns, é para guardar números inteiros com uma precisão de 32 bits
Short	Também guarda valores inteiros, porém com capacidade máxima de metade de um <code>Int</code> , pois ocupa somente 16 bits
Byte	Também guarda números inteiros, porém seu número máximo é 127 pois ele ocupa somente 8 bits
String	É um tipo para guardar valores em texto, uma frase, um nome ou qualquer valor que seja um texto
Char	É um tipo para guardar um único caractere, diferente da <code>String</code> , em que você pode guardar textos
Boolean	É usado para valores booleanos, ou seja, valores que são verdadeiro (<code>True</code>) ou falso (<code>False</code>)

Sintaxe básica do Kotlin

- Tipos de dados:
 - Diferentemente do Java, Kotlin trata os tipos de forma unificada.
 - Na prática, valores como Int, Double e Boolean possuem propriedades e métodos.
 - O compilador utiliza tipos primitivos da JVM quando possível, mantendo a eficiência.

```
fun main() {  
    val x: Int = 10;  
    println(x)  
    println(x::class)  
    var y: Double = x.toDouble()  
    println(y)  
    println(y::class)  
    var z: Float = x.toFloat()  
    println(z)  
    println(z::class)  
    var a: String = x.toString()  
    println(a)  
    println(a::class)  
}
```



```
10  
int (Kotlin reflection is not available)  
10.0  
double (Kotlin reflection is not available)  
10.0  
float (Kotlin reflection is not available)  
10  
class java.lang.String (Kotlin reflection is not available)
```

Sintaxe básica do Kotlin

- Tipos numéricos: similares aos suportados pelo Java.

Tipo	Bit
Double	64
Float	32
Long	64
Int	32
Short	16
Byte	8

Sintaxe básica do Kotlin

- Tipos numéricos: similares aos suportados pelo Java.

Tipo	Bit
Double	64
Float	32
Long	64
Int	32
Short	16
Byte	8

```
val numero = 10  
  
println(numero.toString())  
println(numero.inc())  
println(numero.dec())
```

Sintaxe básica do Kotlin

- Literais Numéricos:
 - Kotlin permite utilizar sublinhado (_) em valores numéricos para melhorar a legibilidade do código.
 - O sublinhado é ignorado pelo compilador.

```
fun main() {  
    val umMilhao = 1000000  
    print(umMilhao)  
}
```



```
fun main() {  
    val umMilhao = 1_000_000  
    print(umMilhao)  
}
```



1000000

Sintaxe básica do Kotlin

- Strings:
 - Kotlin utiliza aspas duplas (" ") para definir strings.
 - Strings são objetos e possuem propriedades e métodos.

```
val texto = "Boa tarde, seja bem-vindo ao sistema"  
print(texto)
```



```
Boa tarde, seja bem-vindo ao sistema
```

Sintaxe básica do Kotlin

- Templates de Strings:
 - Permitem inserir expressões e variáveis diretamente dentro de uma String.
 - Utilizam o símbolo \$ para interpolação.
 - Tornam o código mais simples e legível, evitando concatenações manuais.

```
val nomeUsuario = "Lucas"  
println("Bem-vindo, $nomeUsuario!")
```



```
Bem-vindo, Lucas!
```

Sintaxe básica do Kotlin

- Strings de Múltiplas Linhas:
 - Kotlin permite criar strings com várias linhas utilizando três aspas duplas (""").
 - Mantém quebras de linha e formatação do texto.
 - Útil para textos longos, JSON, HTML, SQL, entre outros.
 - `trimIndent()`: remove os espaços antes de cada linha da string

```
val mensagem = """  
    Olá, Lucas!  
    Bem-vindo ao Kotlin.  
    Esta é uma string  
    de múltiplas linhas.  
    """.trimIndent()  
  
println(mensagem)
```



```
Olá, Lucas!  
Bem-vindo ao Kotlin.  
Esta é uma string  
de múltiplas linhas.
```


Sintaxe básica do Kotlin

- Char:
 - O tipo Char representa um único caractere.
 - A atribuição utiliza aspas simples (' ').

```
val letra: Char = 'A'  
val simbolo = '#'
```

Sintaxe básica do Kotlin

- Tipo Boolean:
 - O tipo Boolean representa valores lógicos: verdadeiro (true) ou falso (false).
 - As operações booleanas utilizam operadores lógicos.

Operação	Operador
E	&&
OU	
Não	!

Sintaxe básica do Kotlin

- Tipo Boolean:
 - O tipo Boolean representa valores lógicos: verdadeiro (true) ou falso (false).
 - As operações booleanas utilizam operadores lógicos.

Operação	Operador
E	&&
OU	
Não	!

```
val idade = 20
val possuiCarteira = true

val podeDirigir = idade >= 18 && possuiCarteira
```

Sintaxe básica do Kotlin

- Operações com Boolean:
 - O tipo Boolean possui funções para operações lógicas.
 - As principais operações são:
 - `and()` → E lógico
 - `or()` → OU lógico
 - `not()` → Negação lógica

```
val a = true
val b = false

println(a.and(b)) // false
println(a.or(b))  // true
println(a.not())  // false
```

Sintaxe básica do Kotlin

- Estruturas de decisão: **if-else**
 - Permite executar diferentes blocos de código de acordo com uma condição.
 - A condição deve retornar um valor booleano (true ou false).
 - Diferentemente do Java, o if em Kotlin é uma expressão e pode retornar um valor.

```
val idade = 20

if (idade >= 18) {
    println("Maior de idade")
} else {
    println("Menor de idade")
}
```



```
val idade = 20

if (idade >= 18)
    println("Maior de idade")
else
    println("Menor de idade")
```

Sintaxe básica do Kotlin

- Estruturas de decisão: **if-else**
 - Permite executar diferentes blocos de código de acordo com uma condição.
 - A condição deve retornar um valor booleano (true ou false).
 - Diferentemente do Java, o if em Kotlin é uma expressão e pode retornar um valor.

```
val idade = 20

val mensagem = if (idade >= 18)
    "Maior de idade"
else
    "Menor de idade"
```

Sintaxe básica do Kotlin

- Estruturas de decisão: **if-else**
 - Permite executar diferentes blocos de código de acordo com uma condição.
 - A condição deve retornar um valor booleano (true ou false).
 - Diferentemente do Java, o if em Kotlin é uma expressão e pode retornar um valor.

```
val x = 10
val y = 20

val resultado = if (x > y) "x é maior" else "y é maior"
```

Sintaxe básica do Kotlin

- Estruturas de Decisão: **if - else if - else**
 - Permite avaliar múltiplas condições em sequência.
 - O primeiro bloco cuja condição for verdadeira será executado.
 - O bloco else é opcional e executa quando nenhuma condição é atendida.
 - É expressão e pode retornar um valor.

```
val nota = 8

if (nota >= 9) {
    println("Excelente")
} else if (nota >= 7) {
    println("Aprovado")
} else if (nota >= 5) {
    println("Recuperação")
} else {
    println("Reprovado")
}
```


Sintaxe básica do Kotlin

- Estruturas de Decisão: **when**
 - **when** substitui o switch-case de linguagens como Java.
 - Executa blocos de código conforme o valor de uma expressão.
 - Mais flexível, permite intervalos, condições e múltiplos valores.
 - Assim como **if**, é uma expressão e pode retornar valores.

```
val dia = 3

when (dia) {
    1 -> println("Domingo")
    2 -> println("Segunda-feira")
    3 -> println("Terça-feira")
    else -> println("Dia inválido")
}
```

Sintaxe básica do Kotlin

- Estruturas de Decisão: **when**
 - when substitui o switch-case de linguagens como Java.
 - Executa blocos de código conforme o valor de uma expressão.
 - Mais flexível, permite intervalos, condições e múltiplos valores.
 - Assim como if, é uma expressão e pode retornar valores.

```
when (nota) {  
    0, 1, 2, 3 -> println("Insuficiente")  
    4, 5, 6 -> println("Regular")  
    7, 8, 9, 10 -> println("Aprovado")  
}
```

Sintaxe básica do Kotlin

- Estruturas de Decisão: **when**
 - **when** substitui o switch-case de linguagens como Java.
 - Executa blocos de código conforme o valor de uma expressão.
 - Mais flexível, permite intervalos, condições e múltiplos valores.
 - Assim como **if**, é uma expressão e pode retornar valores.

```
when (nota) {  
    in 0..3 -> println("Insuficiente")  
    in 4..6 -> println("Regular")  
    in 7..10 -> println("Aprovado")  
}
```

Sintaxe básica do Kotlin

- Estruturas de Decisão: **when**
 - **when** substitui o switch-case de linguagens como Java.
 - Executa blocos de código conforme o valor de uma expressão.
 - Mais flexível, permite intervalos, condições e múltiplos valores.
 - Assim como **if**, é uma expressão e pode retornar valores.

```
val resultado = when (nota) {  
    10 -> "Nota máxima"  
    else -> "Continue estudando"  
}
```

Sintaxe básica do Kotlin

- Arrays:
 - São coleções indexadas de elementos do mesmo tipo (estrutura homogênea).
 - Possuem tamanho fixo, definido no momento da criação.
 - Seus elementos podem ser acessados por meio de índices.
 - Diferentemente de listas, o tamanho de um array não pode ser alterado após sua criação.

```
val numeros = arrayOf(10, 20, 30)  
println(numeros[0]) // 10
```

Sintaxe básica do Kotlin

- Arrays:
 - Arrays são mutáveis, mesmo quando declarados com `val`.
 - O `val` impede a troca da referência, mas não a alteração dos elementos.
 - O Kotlin também possui arrays especializados (`IntArray`, `DoubleArray`, etc.), para evitar a conversão dos tipos primitivos em objetos.

```
val idades = intArrayOf(20, 25, 30)
val alturas = doubleArrayOf(1.70, 1.80, 1.65)
val notas = floatArrayOf(7.5f, 8.0f, 9.5f)
val ativos = booleanArrayOf(true, false, true)
```


Sintaxe básica do Kotlin

- Listas:
 - Utilizadas para armazenar coleções de elementos acessados por índices.
 - Kotlin possui duas interfaces principais:
 - List: lista somente leitura.
 - MutableList: possui tamanho dinâmico e permite alterações.
 - O acesso aos elementos utiliza índices, assim como em arrays.

Sintaxe básica do Kotlin

- Definindo uma lista mutável:

```
val nomes = mutableListOf("Ana", "João", "Maria")
```

- Para criar uma lista mutável vazia em Kotlin:

```
val numeros = mutableListOf<Int>()
```

Sintaxe básica do Kotlin

- O que acontece se eu criar a seguinte lista?

```
val dados = mutableListOf("Ana", 30, true, 5.5)
```



Sintaxe básica do Kotlin

- O que acontece se eu criar a seguinte lista?

```
val dados = mutableListOf("Ana", 30, true, 5.5)
```

- Kotlin identifica que os elementos possuem tipos diferentes.
- A lista é criada como `MutableList<Any>`.
- `Any` é o tipo mais genérico da hierarquia de classes do Kotlin.
- `Any` é o equivalente ao `Object` do Java

Sintaxe básica do Kotlin

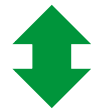
- Adicionando elementos em uma Lista:

```
val frutas = mutableListOf("Maçã", "Banana")  
  
frutas.add("Uva") // Adiciona ao final  
frutas.add(1, "Laranja") // Adiciona na posição 1  
frutas.addAll(listOf("Pera", "Kiwi")) // Adiciona vários elementos
```

Sintaxe básica do Kotlin

- Acessando o primeiro e o último elemento de uma lista:

```
val frutas = mutableListOf("Maçã", "Banana", "Uva", "Pera")  
  
println(frutas[0])           // Primeiro elemento  
println(frutas[frutas.size - 1]) // Último elemento
```



```
val frutas = mutableListOf("Maçã", "Banana", "Uva", "Pera")  
  
println(frutas.first()) // Primeiro elemento  
println(frutas.last())  // Último elemento
```

Sintaxe básica do Kotlin

- Removendo elementos em uma Lista:

```
val frutas = mutableListOf("Maçã", "Banana", "Uva", "Pera", "Uva")

frutas.remove("Banana")           // Remove pelo valor
frutas.removeAt(0)                 // Remove pelo índice
frutas.removeFirst()              // Remove o primeiro elemento
frutas.removeLast()               // Remove o último elemento
frutas.removeAll(listOf("Uva", "Pera")) // Remove vários elementos
frutas.clear()                    // Remove todos os elementos
```


Sintaxe básica do Kotlin

- Verificando o tamanho e se um elemento está presente em uma lista:

```
val frutas = mutableListOf("Maçã", "Banana", "Uva")

println(frutas.contains("Banana")) // true
println("Laranja" in frutas)      // false

println(frutas.size)               // 3
```

Sintaxe básica do Kotlin

- Função `filter()`:
 - Cria uma nova lista contendo apenas os elementos que satisfazem uma condição.
 - A condição é definida por um predicado (expressão que retorna true ou false).
 - A lista original não é modificada.

```
val numeros = listOf(10, 15, 20, 25, 30)  
val pares = numeros.filter { it % 2 == 0 }  
println(pares)
```



```
[10, 20, 30]
```


Sintaxe básica do Kotlin

- Função filter():
 - Cria uma nova lista contendo apenas os elementos que satisfazem uma condição.
 - A condição é definida por um predicado (expressão que retorna true ou false).
 - A lista original não é modificada.

```
val nomes = listOf("Ana", "João", "Maria", "José")  
  
val nomesComJ = nomes.filter { it.startsWith("J") }  
  
println(nomesComJ)
```



```
[João, José]
```

Sintaxe básica do Kotlin

- Função `map()`
 - Cria uma nova lista transformando cada elemento da lista original.
 - Aplica uma operação a todos os elementos.
 - A lista original não é modificada.

```
val numeros = listOf(1, 2, 3, 4, 5)  
  
val quadrados = numeros.map { it * it }  
  
println(quadrados)
```



```
[1, 4, 9, 16, 25]
```

Sintaxe básica do Kotlin

- Função `map()`
 - Cria uma nova lista transformando cada elemento da lista original.
 - Aplica uma operação a todos os elementos.
 - A lista original não é modificada.

```
val nomes = listOf("Ana", "João", "Maria")  
  
val maiusculos = nomes.map { it.uppercase() }  
  
println(maiusculos)
```



```
[ANA, JOÃO, MARIA]
```

Sintaxe básica do Kotlin

- Definindo uma lista imutável:

```
val frutas = listOf("Maçã", "Banana", "Uva")
```

- Listas imutáveis não podem ser modificadas após sua criação.
- Métodos como `add()`, `addAll()`, `remove()`, `removeAt()`, `removeAll()` e `clear()` não estão disponíveis.

Sintaxe básica do Kotlin

- A função `toList()` cria uma lista somente leitura a partir de uma lista mutável.
- A nova lista contém os mesmos elementos da lista original.
- Alterações na lista original não afetam a lista criada por `toList()`.

```
val listaMutavel = mutableListOf("Ana", "João", "Maria")
val listaImutavel = listaMutavel.toList()

listaMutavel.add("Lucas")

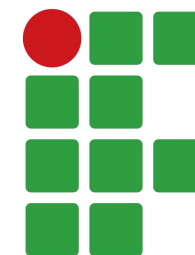
println(listaMutavel)    // [Ana, João, Maria, Lucas]
println(listaImutavel)   // [Ana, João, Maria]
```

Dúvidas



PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco